

⋮

TYPECASTING, DEBUGGER, SCANNER

Coders.Eleven - JAVA



#1 Typecasting

Typkompatibilität

Implizites/Explizites Typecasting

Local Variable Type Inference

#3 Scanner

Motivation

Anlegen

Methoden

#2 Debugger

Vorgehensweise



#1 Typecasting

Typkompatibilität
Implizites/Explizites Typecasting
Local Variable Type Inference



Ausdrücke & mehrere Datentypen

- Variablen unterschiedlichen Typs in einem Ausdruck
 - Welchen Typ hat der Ausdruck?
- Für alle arithmetischen Operationen gilt:
 - Der „kleinere“ Operand wird vor der Ausführung der Operation in den „größeren“ konvertiert
 - Der Ausdruck bekommt dann den gleichen Typ wie seine Operanden
- Beispiel:

```
int a = 12345;  
long b = 54321L;  
  
long c = a + b;  
  
System.out.println(c);
```

a + b ist vom Typ long

Typkompatibilität

- Operanden eines Ausdrucks müssen kompatibel sein
- Beispiel für ein Problem:

```
int x;  
x = 2.5;
```

- Zwei Typen sind kompatibel, wenn
 - sie gleich sind,
 - wenn sie durch implizite Typanpassung zum gleichen Typ führen

Implizite Typanpassung

- Automatische Typanpassung
- Nur in eine Richtung
- byte → short → int → long → float → double bzw. char → int → long → float → double

Explizite Typanpassung (Cast)

- Form: (Zieltyp) Ausdruck
- Beispiel von vorher:

```
int x;  
x = (int) 2.5;
```

- Kann zu Datenverlusten führen!

Beispiel

- Was passiert hier?

```
int higherType = 130;  
byte smallerType = (byte) higherType;  
  
System.out.println(smallerType);
```

-126

- Achtung: Werte können verloren gehen!

In unserem Beispiel: Der Wertebereich von byte geht von -128 bis 127. Die überschüssigen Werte werden von hinten aufgezählt.

Local Variable Type Inference

- Keine explizite Typangabe auf der linken Seite einer Variablendeklaration
 - Nur bei lokalen Variablen
 - Compiler muss anhand der rechten Seite den konkreten Typ ermitteln können
- Beispiel:

```
var a = 10;  
var b = 20.5;  
var c = "Hello";  
  
System.out.println(a + ", " + b + ", " + c);
```

10, 20.5, Hello

#2 Debugger

Vorgehensweise



Debugging

- Ausgangspunkt: Programm hat einen Fehler
 - Ziel: Programm ohne Fehler
- Möglichkeiten:
 - Systematische Überlegungen
 - `print`-Anweisungen
 - Debugger der IDE

Vorgehensweise

- Daten analysieren (Testresultate, Ausgaben, Fehlermeldungen, Programmtext)
- Hypothese aufstellen
- Wiederholbare Experimente festlegen
- Einfachste Eingabe für Tests festlegen

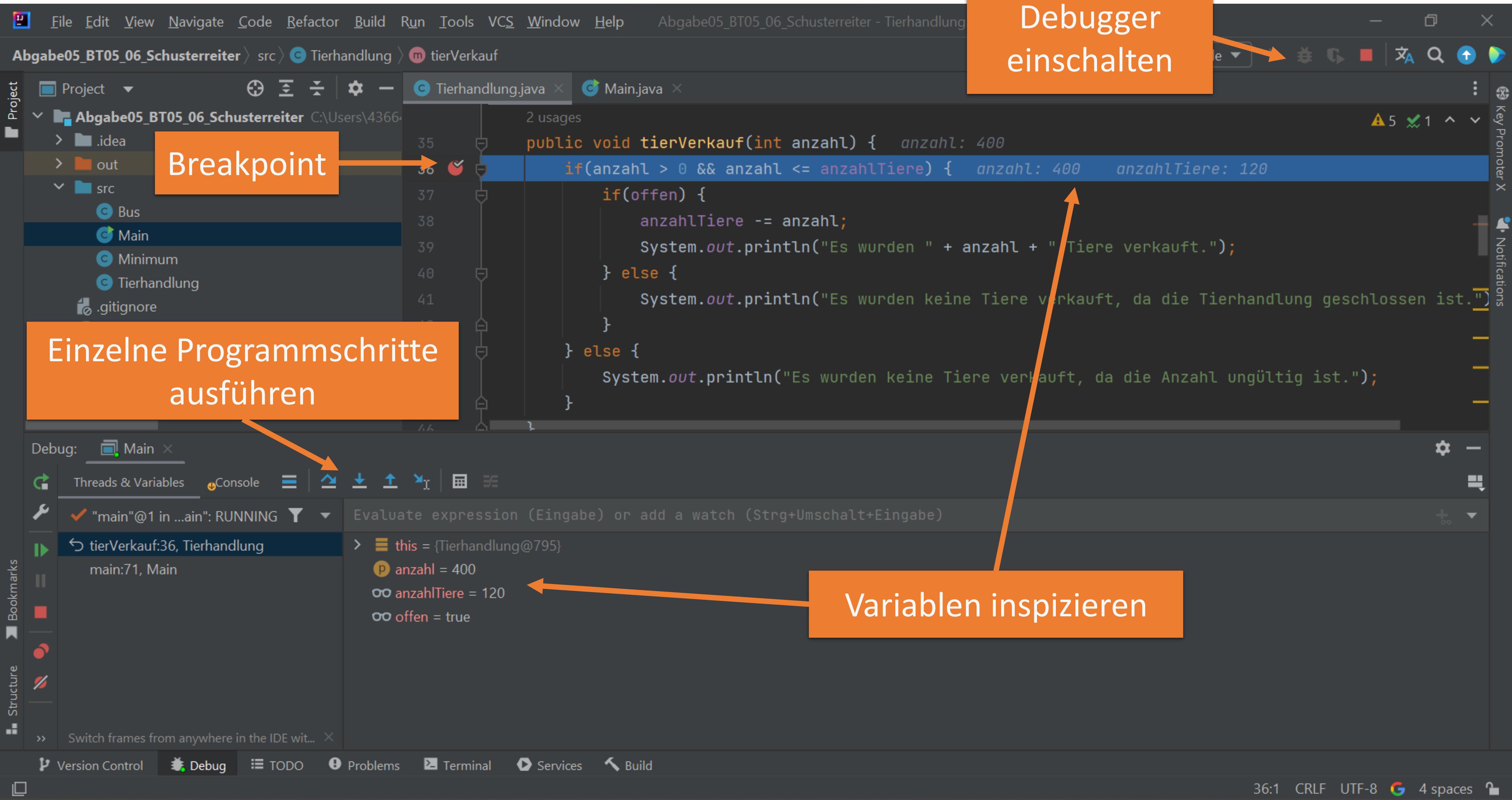
print-Anweisungen

- Einfachste Methode zum Überprüfen von Hypothesen
- Typische Punkte für Ausgaben
 - Betreten einer Methode
 - Parameterwerte
 - Rückgabewerte
 - Direkt vor und nach einer Schleife – schauen wir uns später an
- In großen Programmen:
 - Problemzone mithilfe der „binären Suche“ eingrenzen
 - `println`-Anweisung nach der Hälfte des Programms
 - Abhängig von der Ausgabe neu in eine Hälfte platzieren usw.
 - Meist umständlich

Debugger

- Werkzeug zum Diagnostizieren und Auffinden von Fehlern in Programmen
- Typische Funktionen
 - Steuerung des Programmablaufs (Breakpoints)
 - Schrittweise Durchführung von Programmen
 - Inspizieren und Modifizieren von Variablen

Debugger in IntelliJ



Fehler finden

- Einfache Fehler:
 - Indexfehler
 - NullPointerException (NPE)
 - Falsches Format bei Eingaben
- Schwierige Fehler:
 - Logische Fehler
 - Hilfreich:
 - Diagramme zeichnen
 - Programm anderen Personen erklären

Debuggen größerer Programme

- Schlecht:
 - Ganzes Programm schreiben, testen, debuggen
- Besser:
 - Einzelne Methode implementieren, testen, debuggen, usw.
 - Am Ende alle Methoden integrieren
- Schlecht:
 - Programm ändern, Änderungen merken, testen, Wo war der Fehler nochmal?
- Besser:
 - Backup des Programms (Versionsverwaltung), Programm ändern, Fehler dokumentieren, testen, Versionen vergleichen

#3 Scanner

Motivation

Anlegen

Methoden



Motivation

- Variablen immer mit fixen oder zufälligen Werten belegen ist nicht praxistauglich
- Oft möchte man Daten selbst eingeben
- Unterschiedliche Möglichkeiten
 - Wir verwenden den Scanner

Scanner

- Einlesen über die Kommandozeile
 - Geschieht über Inputstream System.in
- Scanner unterteilt Input in Tokens
- Tokens
 - Primitive Datentypen oder Strings
 - Standardmäßig durch Leerzeichen getrennt
 - Beispiel:

Das ist ein String

4 Tokens durch drei
Leerzeichen getrennt



Anlegen eines Scanners

- Scanner-Klasse benutzen
 - Klasse Scanner muss dafür importiert werden

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
    }
}
```

Genaue Bezeichnung des Scanners

System.in = Eingabe auf der Konsole

Objekt der Klasse Scanner anlegen

Beispiel

```
import java.util.Scanner;
```

Import der Scanner-Klasse

```
public class Main {
```

```
    public static void main(String[] args) {
```

Instanz der Scanner-Klasse
erzeugen

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Geben Sie einen ganzzahligen Wert ein: ");
```

```
        int i = sc.nextInt();
```

Einlesen der Konsoleneingabe mit
den Methoden der Scanner-Klasse

```
        System.out.println(i);
```

```
        sc.close();
```

Schließen des Scanners und
des Input-Streams

```
    }
```

```
}
```


Scanner-Methoden zum Einlesen

Datentyp	Token einlesen	Token überprüfen
boolean	nextBoolean()	hasNextBoolean()
byte	nextByte()	hasNextByte()
short	nextShort()	hasNextShort()
int	nextInt()	hasNextInt()
float	nextFloat()	hasNextFloat()
long	nextLong()	hasNextLong()
double	nextDouble()	hasNextDouble()
char	next().charAt(0)	hasNext()
String	next() oder nextLine()	hasNext() oder hasNextLine()

Tipps

- Scanner ist ein bereits bekannter Klassenname („Schlüsselwort“)
 - Keine Variablen und Klassen mit diesem Namen wählen
- Innerhalb einer Klasse wird nur ein Scanner gebraucht
 - Eine Instanz kann öfter verwendet werden